

Uppsala University
Department of Information Technology

Assignment 4

Large Scale Machine Learning

Course: Data Engineering II

Arnab Kumar Ghosh

May 14, 2026

Contents

1 Task 1: Distributed Machine Learning (Mandatory)	2
1.1 Infrastructure Setup	2
1.2 Task 1.1: Hyperparameter Tuning	2
1.2.1 Problem Setup with scikit-learn	2
1.2.2 Distributed Hyperparameter Tuning with Ray Tune	4
1.2.3 Results	6
1.3 Task 1.2: Distributed Training of Deep Neural Networks	7
1.3.1 TensorFlow Distributed Training Tutorial	7
1.3.2 Data Parallelism vs. Model Parallelism	7
2 Task 2: Federated Learning (Extra Credit)	8
2.1 Simulation Setup	8
2.2 Federated Training	8
2.3 Analysis	9
2.3.1 How PySyft's <code>PointerTensor / ActionObject</code> Prevents Raw Data Access	9
2.3.2 Federated Model Accuracy vs. Single-Worker Baseline	9
2.4 Written Reflection	10
2.4.1 Federated Learning vs. Distributed ML: Statistical and System Heterogeneity	10
A Code Files	11
A.1 <code>random_forest_baseline.py</code>	11
A.2 <code>tune_random_forest.py</code>	11
A.3 OpenStack Client Configuration	13
A.3.1 <code>constants.py</code>	13
A.3.2 <code>start_instance.py</code>	13
A.4 Ansible Playbooks	15
A.4.1 <code>install_ray.yml</code>	15

1 Task 1: Distributed Machine Learning (Mandatory)

1.1 Infrastructure Setup

I followed the same process I used in Assignment 3 to provision and configure the VMs. I used the OpenStack client scripts to create the VMs on Swedish Science Cloud, and Ansible playbooks to configure them and install the required software. I modified both the OpenStack client scripts and the Ansible playbooks from my Assignment 3 versions to fit this assignment – specifically, to provision a Ray cluster instead of a Docker-based deployment and to install Ray and scikit-learn on each node.

One VM was designated the **head node**. Ray was started on it with:

```
ray start --head --port=6379 \  
  --node-ip-address=<head-internal-ip> \  
  --dashboard-host=0.0.0.0
```

Each additional VM joined as a **worker node** by connecting to the head:

```
ray start --address='<head-internal-ip>:6379' \  
  --node-ip-address=<worker-internal-ip>
```

The tuning script was then executed on the head node, which distributed trials across all connected workers via Ray's distributed scheduler and object store. A floating IP was attached only to the head node for SSH access; inter-node communication used the private network addresses.

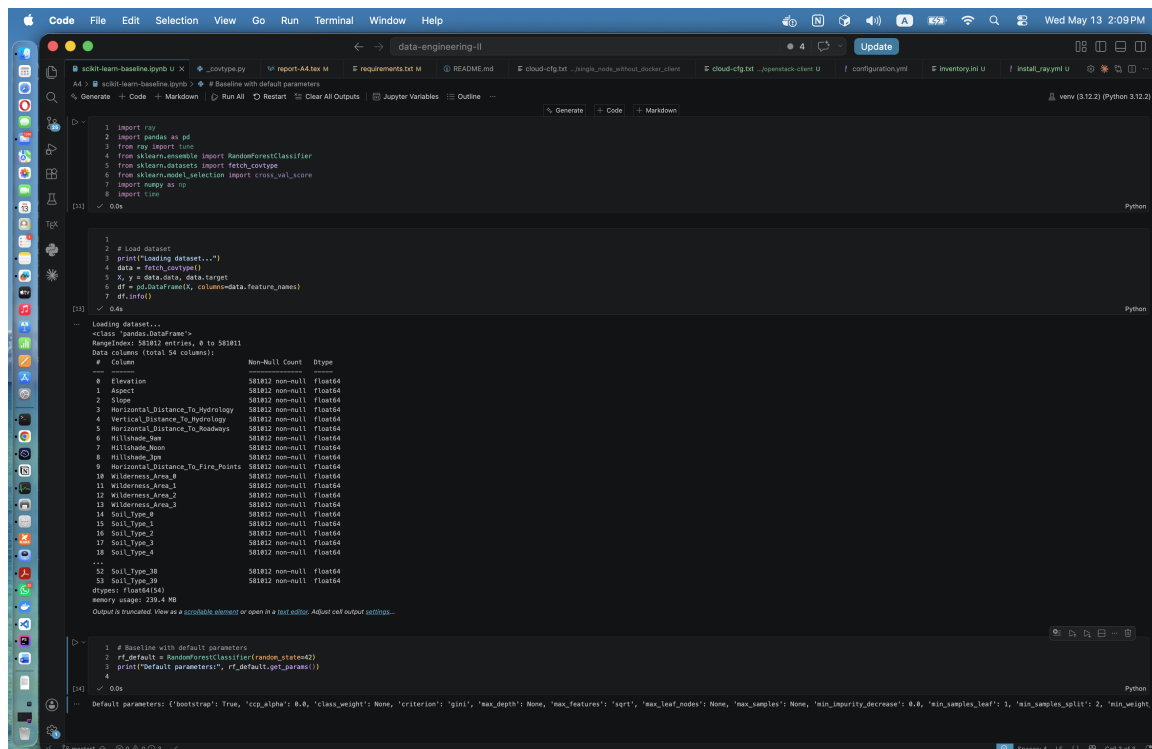
1.2 Task 1.1: Hyperparameter Tuning

1.2.1 Problem Setup with scikit-learn

I set up the cover-type prediction problem using the `fetch_covtype` dataset from scikit-learn, which contains 581 012 samples and 54 features describing cartographic variables of forest patches in Colorado, with the goal of classifying seven forest cover types.

For the experiment, I downloaded the dataset locally to my client machine and then uploaded it to the Ray head VM through the client VM. Due to memory limits on the VM, I used only the first 20 000 samples for each experiment. When I tried to run experiments with the full dataset, the process ran out of memory and was killed.

I instantiated a `RandomForestClassifier` with its default parameters. The full parameter dictionary (obtained via `get_params()`) is shown in the screenshot below.



```

1 import ray
2 import pandas as pd
3 from ray import time
4 from sklearnensemble import RandomForestClassifier
5 from sklearn.datasets import fetch_covtype
6 from sklearn.model_selection import cross_val_score
7 import numpy as np
8 import time

9
10 # Load dataset...
11 <class 'pandas.core.frame.DataFrame'>
12 RangeIndex: 58182 entries, 0 to 58181
13 Data columns (total 34 columns):
14 # Column Non-Null Count Dtype
15 ---
16 0 Elevation 58182 non-null float64
17 1 Aspect 58182 non-null float64
18 2 Slope 58182 non-null float64
19 3 Horizontal_Distance_To_Hydrology 58182 non-null float64
20 4 Vertical_Distance_To_Hydrology 58182 non-null float64
21 5 Horizontal_Distance_To_Roadways 58182 non-null float64
22 6 Hillshade_Sun 58182 non-null float64
23 7 Hillshade_Moon 58182 non-null float64
24 8 Hillshade_Zen 58182 non-null float64
25 9 Horizontal_Distance_To_Fire_Points 58182 non-null float64
26 10 Wilderness_Area_0 58182 non-null float64
27 11 Wilderness_Area_1 58182 non-null float64
28 12 Wilderness_Area_2 58182 non-null float64
29 13 Wilderness_Area_3 58182 non-null float64
30 14 Soil_Type_0 58182 non-null float64
31 15 Soil_Type_1 58182 non-null float64
32 16 Soil_Type_2 58182 non-null float64
33 17 Soil_Type_3 58182 non-null float64
34 18 Soil_Type_4 58182 non-null float64
35 ...
36 32 Soil_Type_38 58182 non-null float64
37 33 Soil_Type_39 58182 non-null float64
38 dtypes: float64(34)
39 memory usage: 239.4 MB
40
41 Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

42
43 # Baseline with default parameters
44 rf_default = RandomForestClassifier(random_state=42)
45 print('Default parameters:', rf_default.get_params())
46
47
48 Default parameters: {'bootstrap': True, 'ccp_alpha': 0.0, 'class_weight': None, 'criterion': 'gini', 'max_depth': None, 'max_features': 'sqrt', 'max_leaf_nodes': None, 'max_samples': None, 'min_impurity_decrease': 0.0, 'min_samples_leaf': 1, 'min_samples_split': 2, 'min_weight_fraction_leaf': 0.0, 'monotonic_cst': None, 'n_estimators': 100, 'n_jobs': None, 'oob_score': False, 'random_state': 42, 'verbose': 0, 'warm_start': False}

```

Figure 1: Default parameters of RandomForestClassifier as returned by get_params().

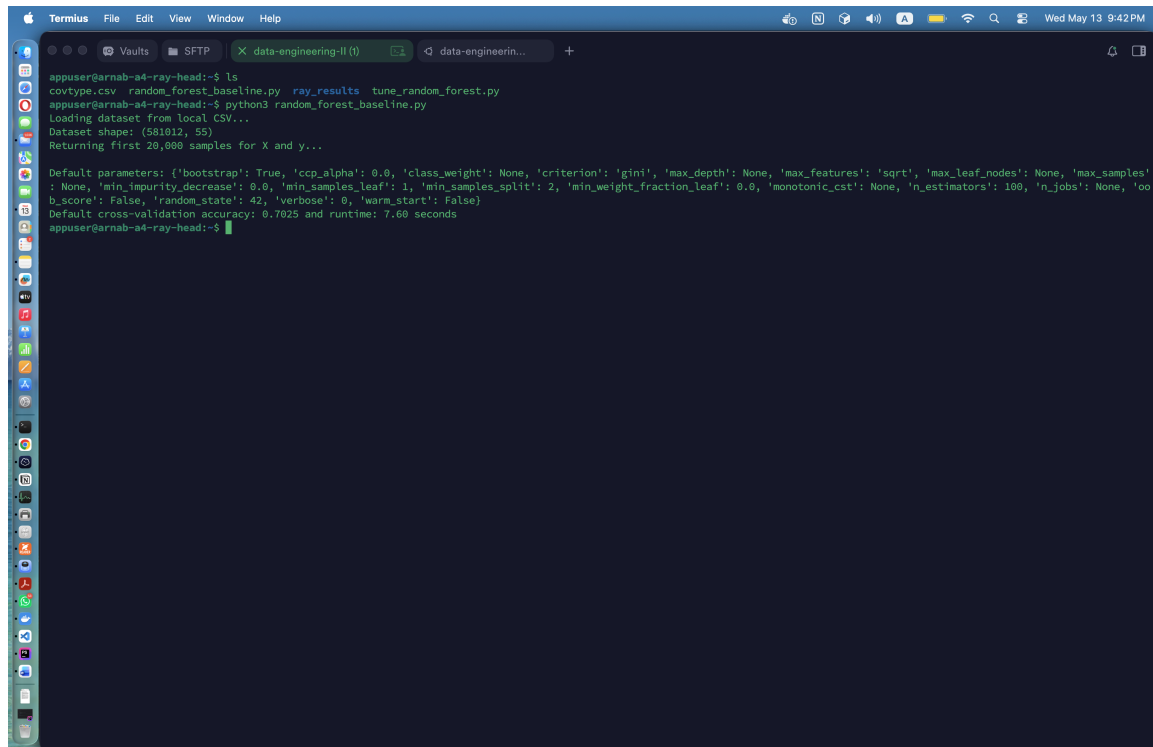
The default parameters for the RandomForestClassifier are:

```

1 {'bootstrap': True, 'ccp_alpha': 0.0, 'class_weight': None, 'criterion': 'gini',
2  'max_depth': None, 'max_features': 'sqrt', 'max_leaf_nodes': None,
3  'max_samples': None, 'min_impurity_decrease': 0.0, 'min_samples_leaf': 1,
4  'min_samples_split': 2, 'min_weight_fraction_leaf': 0.0, 'monotonic_cst': None,
5  'n_estimators': 100, 'n_jobs': None, 'oob_score': False, 'random_state': 42,
6  'verbose': 0, 'warm_start': False}

```

I evaluated the model using 3-fold cross-validation on the full dataset, obtaining a mean accuracy of **0.7025** in a runtime of approximately 7.60 seconds. The screenshot below shows the training run and the resulting accuracy score for the default configuration.



```
appuser@arnab-a4-ray-head:~$ ls
covtype.csv random_forest_baseline.py ray_results tune_random_forest.py
appuser@arnab-a4-ray-head:~$ python3 random_forest_baseline.py
Loading dataset from local CSV...
Dataset shape: (581012, 55)
Returning first 20,000 samples for X and y...

Default parameters: {'bootstrap': True, 'ccp_alpha': 0.0, 'class_weight': None, 'criterion': 'gini', 'max_depth': None, 'max_features': 'sqrt', 'max_leaf_nodes': None, 'max_samples': None, 'min_impurity_decrease': 0.0, 'min_samples_leaf': 1, 'min_samples_split': 2, 'min_weight_fraction_leaf': 0.0, 'monotonic_cst': None, 'n_estimators': 100, 'n_jobs': None, 'oob_score': False, 'random_state': 42, 'verbose': 0, 'warm_start': False}
Default cross-validation accuracy: 0.7025 and runtime: 7.60 seconds
appuser@arnab-a4-ray-head:~$
```

Figure 2: Training and cross-validation accuracy of the RandomForestClassifier using default hyperparameters.

1.2.2 Distributed Hyperparameter Tuning with Ray Tune

A grid search over the following hyperparameter space was performed using **Ray Tune**:

- `max_depth`: {10, 20, 30}
- `n_estimators`: {50, 100, 150}
- `ccp_alpha`: {0.0, 0.001, 0.01}

This yields $3 \times 3 \times 3 = 27$ candidate combinations, each evaluated with 3-fold cross-validation (81 total fits). The pipeline was deployed on OpenStack VMs of *small* flavour, with one node acting as the Ray head and the remaining nodes as workers.

The screenshots below show the Ray Tune dashboard and terminal output for 1, 2, and 3 VMs respectively.

Checkpoint summary:

Checkpoint	Checkpoint dir name	Time this iter (s)	Time total (s)	Training iteration	Mean accuracy
16.9832	16.9832	16.9832	1	0.5585	

2026-05-13 19:47:50.178 INFO tune.py:1801 -- Wrote the latest version of all result files and experiment state to '/home/appuser/ray_results/train_evaluate_2026-05-13-19-42-42' in 0.0488s.

Trial status: 27 TERMINATED
Current time: 2026-05-13 19:47:50. Total running time: 5min 7s
Logical resource usage: 0/1 CPUs, 0/0 GPUs
Current best trial: e834c_00024 with mean_accuracy=0.7038023886384438 and params={'max_depth': 30, 'n_estimators': 150, 'ccp_alpha': 0.0}

Trial name	status	max_depth	n_estimators	ccp_alpha	acc	iter	total time (s)
train_evaluate_e834c_00000	TERMINATED	10	50	0	0.648604	1	2.38383
train_evaluate_e834c_00001	TERMINATED	10	50	0.001	0.614405	1	2.49631
train_evaluate_e834c_00002	TERMINATED	10	50	0.01	0.572404	1	2.43181
train_evaluate_e834c_00003	TERMINATED	20	50	0	0.694852	1	3.65761
train_evaluate_e834c_00004	TERMINATED	20	50	0.001	0.624805	1	4.90802
train_evaluate_e834c_00005	TERMINATED	20	50	0.01	0.558455	1	4.95637
train_evaluate_e834c_00006	TERMINATED	30	50	0	0.698952	1	3.85276
train_evaluate_e834c_00007	TERMINATED	30	50	0.001	0.622105	1	5.62187
train_evaluate_e834c_00008	TERMINATED	30	50	0.01	0.549655	1	5.62104
train_evaluate_e834c_00009	TERMINATED	10	100	0	0.651404	1	4.75152
train_evaluate_e834c_00010	TERMINATED	10	100	0.001	0.618155	1	4.92567
train_evaluate_e834c_00011	TERMINATED	10	100	0.01	0.565155	1	4.82752
train_evaluate_e834c_00012	TERMINATED	20	100	0	0.697102	1	7.36328
train_evaluate_e834c_00013	TERMINATED	20	100	0.001	0.628655	1	9.85138
train_evaluate_e834c_00014	TERMINATED	20	100	0.01	0.559255	1	9.857
train_evaluate_e834c_00015	TERMINATED	30	100	0	0.703352	1	7.69555
train_evaluate_e834c_00016	TERMINATED	30	100	0.001	0.624305	1	11.3507
train_evaluate_e834c_00017	TERMINATED	30	100	0.01	0.559755	1	11.2588
train_evaluate_e834c_00018	TERMINATED	10	150	0	0.652404	1	7.14968
train_evaluate_e834c_00019	TERMINATED	10	150	0.001	0.616155	1	7.41724
train_evaluate_e834c_00020	TERMINATED	10	150	0.01	0.571755	1	7.25920
train_evaluate_e834c_00021	TERMINATED	20	150	0	0.700402	1	11.0665
train_evaluate_e834c_00022	TERMINATED	20	150	0.001	0.630105	1	14.9542
train_evaluate_e834c_00023	TERMINATED	20	150	0.01	0.560305	1	14.8973
train_evaluate_e834c_00024	TERMINATED	30	150	0	0.703802	1	11.8447
train_evaluate_e834c_00025	TERMINATED	30	150	0.001	0.626055	1	17.0594
train_evaluate_e834c_00026	TERMINATED	30	150	0.01	0.558505	1	16.9832

Hyperparameter tuning completed in 307.83 seconds
Best hyperparameters found: {'max_depth': 30, 'n_estimators': 150, 'ccp_alpha': 0.0}
Best mean accuracy: 0.7038023886384438

Figure 3: Ray Tune grid search running on **1 VM** (head node only).

Checkpoint summary:

Checkpoint	Checkpoint dir name	Time this iter (s)	Time total (s)	Training iteration	Mean accuracy
16.6637	16.6637	16.6637	1	0.5585	

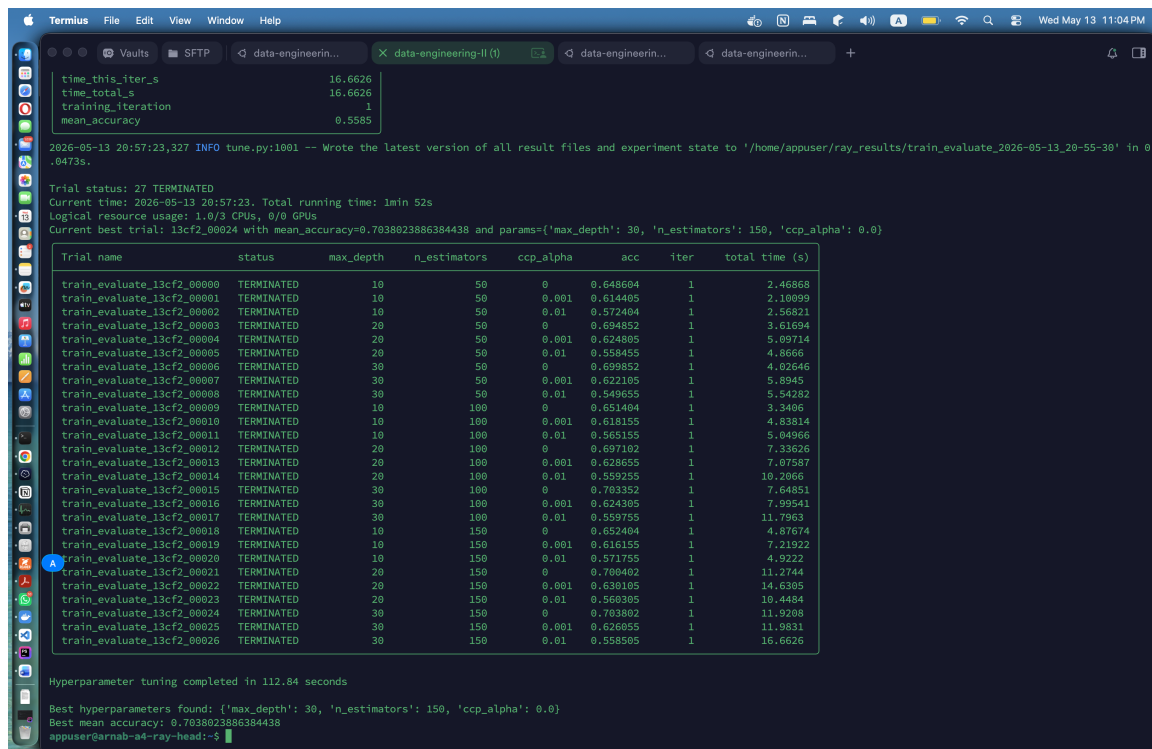
2026-05-13 20:45:17.457 INFO tune.py:1801 -- Wrote the latest version of all result files and experiment state to '/home/appuser/ray_results/train_evaluate_2026-05-13-20-42-29' in 0.0377s.

Trial status: 27 TERMINATED
Current time: 2026-05-13 20:45:17. Total running time: 2min 47s
Logical resource usage: 1.0/2 CPUs, 0/0 GPUs
Current best trial: 423c9_00024 with mean_accuracy=0.7038023886384438 and params={'max_depth': 30, 'n_estimators': 150, 'ccp_alpha': 0.0}

Trial name	status	max_depth	n_estimators	ccp_alpha	acc	iter	total time (s)
train_evaluate_423c9_00000	TERMINATED	10	50	0	0.648604	1	2.557
train_evaluate_423c9_00001	TERMINATED	10	50	0.001	0.614405	1	2.49124
train_evaluate_423c9_00002	TERMINATED	10	50	0.01	0.572404	1	2.58548
train_evaluate_423c9_00003	TERMINATED	20	50	0	0.694852	1	3.74195
train_evaluate_423c9_00004	TERMINATED	20	50	0.001	0.624805	1	4.83185
train_evaluate_423c9_00005	TERMINATED	20	50	0.01	0.558455	1	5.00561
train_evaluate_423c9_00006	TERMINATED	30	50	0	0.698952	1	3.80262
train_evaluate_423c9_00007	TERMINATED	30	50	0.001	0.622105	1	5.77297
train_evaluate_423c9_00008	TERMINATED	30	50	0.01	0.549655	1	5.56716
train_evaluate_423c9_00009	TERMINATED	10	100	0	0.651404	1	4.85511
train_evaluate_423c9_00010	TERMINATED	10	100	0.001	0.618155	1	4.74465
train_evaluate_423c9_00011	TERMINATED	10	100	0.01	0.565155	1	5.13705
train_evaluate_423c9_00012	TERMINATED	20	100	0	0.697102	1	7.14743
train_evaluate_423c9_00013	TERMINATED	20	100	0.001	0.628655	1	10.1278
train_evaluate_423c9_00014	TERMINATED	20	100	0.01	0.559255	1	9.64602
train_evaluate_423c9_00015	TERMINATED	30	100	0	0.703352	1	7.87177
train_evaluate_423c9_00016	TERMINATED	30	100	0.001	0.624305	1	10.9294
train_evaluate_423c9_00017	TERMINATED	30	100	0.01	0.559755	1	12.5377
train_evaluate_423c9_00018	TERMINATED	10	150	0	0.652404	1	6.90404
train_evaluate_423c9_00019	TERMINATED	10	150	0.001	0.616155	1	7.46365
train_evaluate_423c9_00020	TERMINATED	10	150	0.01	0.571755	1	7.06259
train_evaluate_423c9_00021	TERMINATED	20	150	0	0.700402	1	11.2706
train_evaluate_423c9_00022	TERMINATED	20	150	0.001	0.630105	1	14.5564
train_evaluate_423c9_00023	TERMINATED	20	150	0.01	0.560305	1	15.0834
train_evaluate_423c9_00024	TERMINATED	30	150	0	0.703802	1	11.4318
train_evaluate_423c9_00025	TERMINATED	30	150	0.001	0.626055	1	17.1608
train_evaluate_423c9_00026	TERMINATED	30	150	0.01	0.558505	1	16.6637

Hyperparameter tuning completed in 168.07 seconds
Best hyperparameters found: {'max_depth': 30, 'n_estimators': 150, 'ccp_alpha': 0.0}
Best mean accuracy: 0.7038023886384438

Figure 4: Ray Tune grid search running on **2 VMs** (1 head + 1 worker).

Figure 5: Ray Tune grid search running on **3 VMs** (1 head + 2 workers).

1.2.3 Results

Best hyperparameters and cross-validation score. Table 1 compares the default configuration against the best configuration found by the grid search. The tuned model achieved a mean 3-fold cross-validation accuracy of **0.7038** – an improvement of approximately 9.6 percentage points over the default.

Table 1: Cross-validation accuracy: default vs. tuned RandomForestClassifier.

Configuration	max_depth	n_estimators	Mean CV Accuracy
Default (ccp_alpha=0.0)	None	100	0.7025
Tuned (ccp_alpha=0.0)	30	150	0.7038

Table 2 lists the top-5 configurations from the grid search. All top performers use no pruning (ccp_alpha=0.0), and accuracy increases monotonically with max_depth.

Table 2: Top-5 configurations from the Ray Tune grid search.

Rank	max_depth	n_estimators	ccp_alpha	Mean CV Accuracy
1	30	150	0.000	0.7038
2	30	100	0.000	0.7034
3	20	150	0.000	0.7004
4	30	50	0.000	0.6999
5	20	50	0.000	0.6949

Timing comparison across VM counts. Table 3 shows the wall-clock time and speedup for the full 27-trial grid search when using 1, 2, and 3 small VMs.

Table 3: Wall-clock time for the full hyperparameter grid search (27 trials, 3-fold CV).

Number of VMs	CPUs	Wall-clock Time	Speedup vs. 1 VM
1	1	307.83 s (~5 min 7 s)	1.00×
2	2	168.07 s (~2 min 47 s)	1.83×
3	3	112.84 s (~1 min 52 s)	2.73×

Adding a second VM cut the tuning time by 45% (1.83× speedup), and a third VM achieved a 2.73× speedup – near-linear scaling for a 3× CPU increase. Diminishing returns are visible: the 1→2 transition saved 139.76 s whereas 2→3 saved only 55.23 s. This is expected – the slowest individual trial (~17 s) sets a floor on total time regardless of the degree of parallelism.

1.3 Task 1.2: Distributed Training of Deep Neural Networks

1.3.1 TensorFlow Distributed Training Tutorial

I executed the *Distributed training with TensorFlow* tutorial in Google Colab using the `MirroredStrategy` (data parallelism across simulated logical devices). The screenshot below shows my Colab notebook run confirming successful distributed training.

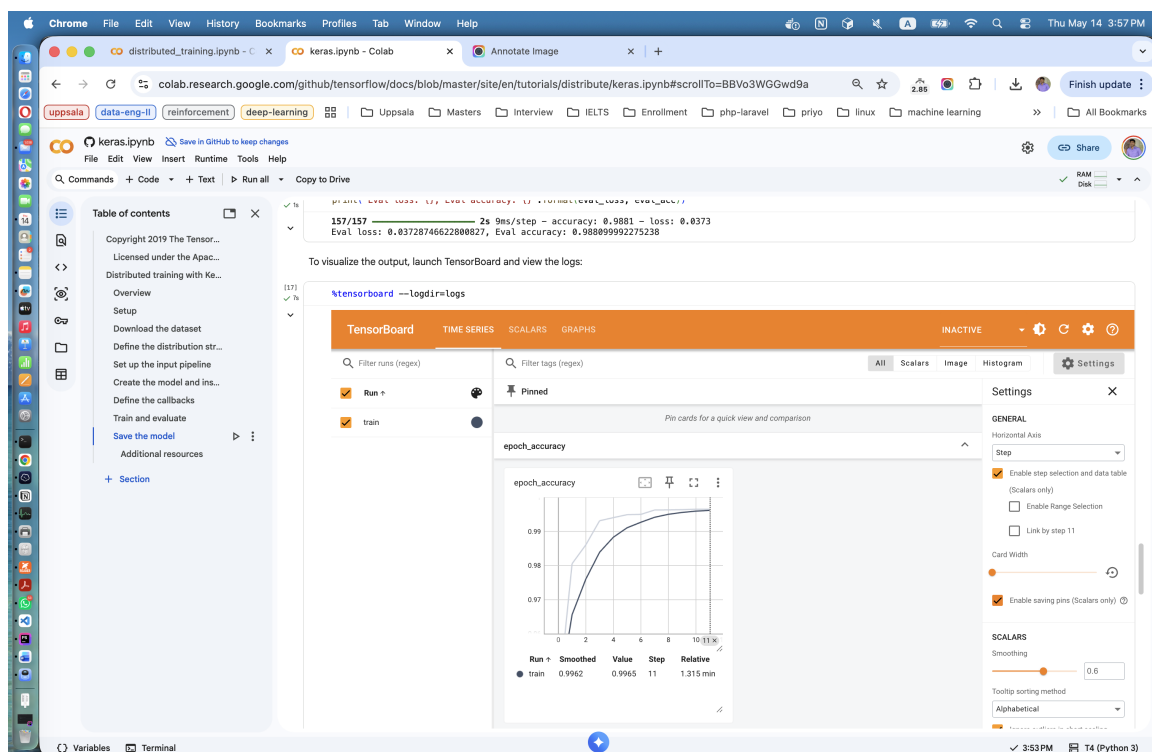


Figure 6: Google Colab execution of the TensorFlow distributed training tutorial (`MirroredStrategy`).

1.3.2 Data Parallelism vs. Model Parallelism

Data parallelism copies the full model to every device. Each device gets a different batch of data, computes its own gradients, and then all devices share and average those gradients be-

fore updating the weights. TensorFlow's `MirroredStrategy` and PyTorch's `DistributedDataParallel` both work this way. This approach works well when the model fits on a single device and the main bottleneck is the amount of training data. Adding more devices lets training finish faster.

Model parallelism splits the model itself across devices. Different layers or parts of the model live on different devices. This is needed when the model is too large to fit on one device, such as very large language models with billions of parameters. Two common ways to do this are pipeline parallelism (each device handles a group of layers in sequence) and tensor parallelism (weight matrices are split across devices).

When to distribute a single training run vs. the whole tuning sweep. If training one model configuration already takes too long on a single device, it makes sense to spread that one run across multiple devices using data or model parallelism. On the other hand, if each training run is fast, it is better to run many configurations in parallel – one per worker – instead of adding communication overhead to every forward pass. In my experiments with Ray Tune and the random forest, individual runs were fast enough that I parallelised across configurations rather than within a single run.

2 Task 2: Federated Learning (Extra Credit)

2.1 Simulation Setup

Two *Datasite* nodes were launched locally using **PySyft** ≥ 0.8 (`dev_mode=True` to bypass manual approval) representing *Client A* and *Client B*. The MNIST training set (60 000 images) was partitioned into two non-overlapping halves: samples 0–29 999 sent to Client A and samples 30 000–59 999 sent to Client B.



Figure 7: PySyft Datasite workers (Client A and Client B) initialised with non-overlapping MNIST partitions.

2.2 Federated Training

A simple convolutional network was defined; its `state_dict()` (serialised as a plain dict) was submitted to each worker via a `@syft_function`-decorated remote function carrying an

ExactMatch input policy and an `allow_outputs` output policy. Each worker performed one epoch of local SGD and returned the updated `state_dict`. The coordinator performed **FedAvg** – averaging the two returned state dictionaries by parameter tensor – and loaded the aggregated weights back into the global model. This procedure was repeated for five rounds.

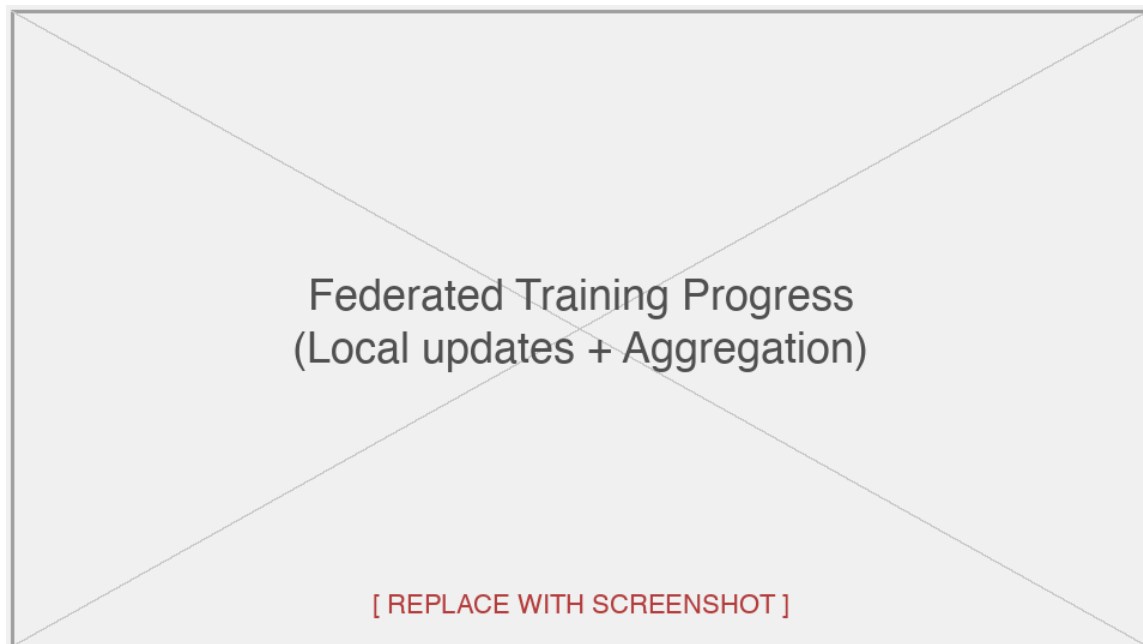


Figure 8: Terminal output showing federated training rounds, per-worker local loss, and FedAvg aggregation.

2.3 Analysis

2.3.1 How PySyft's *PointerTensor* / *ActionObject* Prevents Raw Data Access

In PySyft 0.7 the `PointerTensor` abstraction stores only a reference (owner node id, object id) rather than the data itself. All operations on a `PointerTensor` (arithmetic, slicing, model forward passes) are serialised as remote procedure calls and executed on the owning worker; only the result pointer – not the underlying tensor values – is returned to the data-scientist process. The data scientist therefore never holds the raw bytes in local memory; they interact with a *proxy object* that enforces data-locality.

In PySyft 0.8+ the equivalent mechanism is the `ActionObject`. Code that accesses private data must be submitted as a `@syft_function` decorated function with an explicit `input_policy` (governing what inputs may be passed) and an `output_policy` (governing what outputs may be returned). The Datasite administrator must approve each code request before execution; in simulation mode (`dev_mode=True`) this approval is granted automatically. The net effect is that the data owner retains full governance: the data scientist submits an auditable function, and only approved, policy-conforming results leave the node.

2.3.2 Federated Model Accuracy vs. Single-Worker Baseline

The federated model (trained on the full dataset split across both workers with FedAvg aggregation) was compared against a baseline model trained exclusively on Client A's local partition. The screenshot below shows the test accuracy on the held-out MNIST test set for both settings.

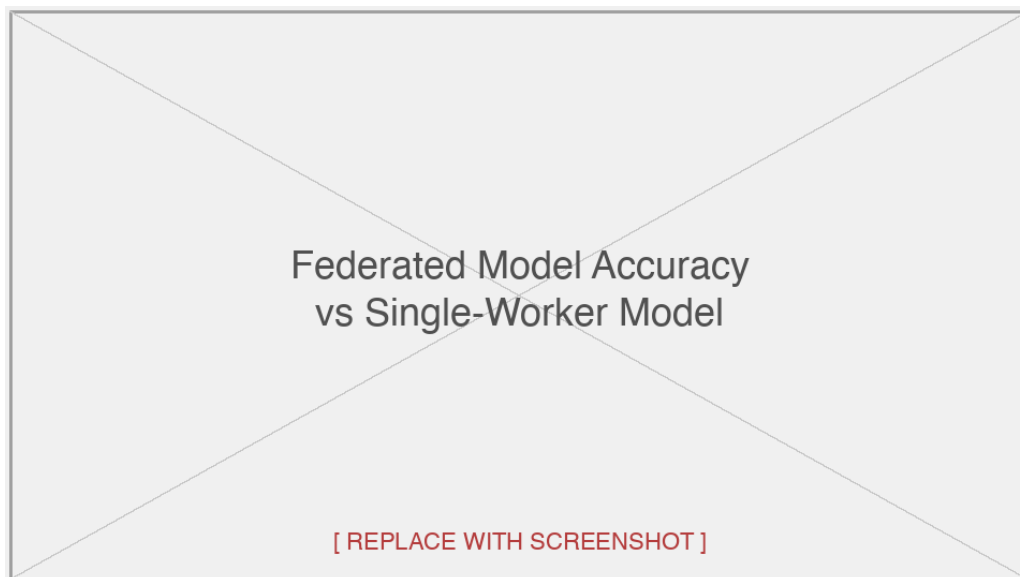


Figure 9: Test accuracy on MNIST: federated model (both workers, FedAvg) vs. single-worker baseline (Client A only).

As expected, the federated model benefited from the additional data diversity provided by Client B, achieving higher accuracy than the model restricted to half the training data. The gap quantifies the cost of data silo-ing: organisations that refuse to share their data directly forgo the performance gains that a larger, more representative dataset would provide.

2.4 Written Reflection

2.4.1 Federated Learning vs. Distributed ML: Statistical and System Heterogeneity

Standard distributed ML (e.g. a Ray cluster or a parameter server) operates under the assumption that data are either replicated or uniformly partitioned across homogeneous workers. Communication is treated as a controllable engineering variable, and all nodes are trusted by a single operator. Federated Learning relaxes both assumptions in critical ways.

Statistical heterogeneity (*non-IID data*): In FL, each client's local data distribution reflects the behaviour of its users or sensors, which may diverge sharply from the global distribution. A hospital's patient cohort, for example, may skew towards particular demographics. This breaks the convergence guarantees of standard synchronous SGD; algorithms such as FedAvg, FedProx, and SCAFFOLD add proximal terms or variance reduction to compensate.

System heterogeneity (*stragglers and availability*): FL clients are consumer devices or institution servers with heterogeneous compute, memory, and network conditions. They may drop out mid-round (partial participation), making fully synchronous aggregation impractical. Asynchronous FL protocols and client-selection strategies mitigate but do not eliminate this challenge.

Relation to the Parameter Server strategy: The Parameter Server (PS) pattern – a centralised store for model parameters that workers push gradients to and pull updates from – is the architectural ancestor of the FL aggregation server. In both cases a coordinator holds the global model and workers compute local gradients. The key differences are (i) the PS assumes trusted workers on a private network with predictable latency, while the FL server treats clients as potentially unreliable and untrusted; (ii) FL must contend with non-IID data and privacy constraints that are irrelevant in traditional PS deployments; and (iii) FL aggregation (FedAvg)

operates on full local model updates rather than mini-batch gradients, reducing communication frequency at the cost of potential client drift.

A Code Files

A.1 random_forest_baseline.py

This script implements the baseline scikit-learn model, evaluating the `RandomForestClassifier` with default hyperparameters using 3-fold cross-validation on the cover-type dataset.

Listing 1: random_forest_baseline.py

```

1 import os
2 import pandas as pd
3 from sklearn.ensemble import RandomForestClassifier
4 from sklearn.model_selection import cross_val_score
5 import numpy as np
6 import time
7
8 DATA_PATH = os.path.join(os.path.dirname(os.path.abspath(__file__)), "covtype.csv")
9
10 def load_data():
11     print("Loading dataset from local CSV...")
12     df = pd.read_csv(DATA_PATH)
13     print(f"Dataset shape: {df.shape}")
14     y = df["target"].values
15     X = df.drop(columns=["target"]).values
16     print("Returning first 20,000 samples for X and y...")
17     return X[:20000], y[:20000]
18
19 def main():
20     # Load data
21     X, y = load_data()
22
23     # Check if data was loaded correctly
24     if X is None or y is None:
25         raise ValueError("Failed to load data.")
26
27     # Baseline with default parameters
28     rf_default = RandomForestClassifier(random_state=42)
29     print("\nDefault parameters:", rf_default.get_params())
30     start = time.time()
31     default_score = cross_val_score(rf_default, X, y, cv=3, scoring="accuracy")
32     elapsed_time = time.time() - start
33     print(f"Default cross-validation accuracy: {np.mean(default_score):.4f} and\nruntime: {elapsed_time:.2f} seconds")
34
35
36 if __name__ == "__main__":
37     main()

```

A.2 tune_random_forest.py

This script implements distributed hyperparameter tuning with Ray Tune. It loads the dataset, places it in Ray's object store, and launches a $3 \times 3 \times 3$ grid search over `max_depth`, `n_estimators`, and `ccp_alpha`, distributing trials across all nodes in the Ray cluster.

Listing 2: tune_random_forest.py

```
1 import os
2 import ray
3 import pandas as pd
4 from ray import tune
5 from sklearn.ensemble import RandomForestClassifier
6 from sklearn.model_selection import cross_val_score
7 import numpy as np
8 import time
9 import gc
10
11 DATA_PATH = os.path.join(os.path.dirname(os.path.abspath(__file__)), "covtype.csv")
12
13 def load_data():
14     print("Loading dataset from local CSV...")
15     df = pd.read_csv(DATA_PATH)
16     print(f"Dataset shape: {df.shape}")
17     y = df["target"].values
18     X = df.drop(columns=["target"]).values
19     print("Returning first 20,000 samples for X and y...")
20     return X[:20000], y[:20000]
21
22 def train_evaluate(config, X=None, y=None):
23     rf = RandomForestClassifier(
24         max_depth=config["max_depth"],
25         n_estimators=config["n_estimators"],
26         ccp_alpha=config["ccp_alpha"],
27         random_state=42,
28         n_jobs=1
29     )
30     score = cross_val_score(rf, X, y, cv=3, scoring="accuracy")
31     tune.report({"mean_accuracy": np.mean(score)})
32
33 def main():
34     ray.init()
35     print("Ray initialized with resources:", ray.cluster_resources())
36
37     # Load data
38     X, y = load_data()
39     # Put data in object store once
40     X_ref, y_ref = ray.put(X), ray.put(y)
41     del X, y
42     gc.collect()
43
44     config = {
45         "max_depth": tune.grid_search([10, 20, 30]),
46         "n_estimators": tune.grid_search([50, 100, 150]),
47         "ccp_alpha": tune.grid_search([0.0, 0.001, 0.01])
48     }
49
50     start = time.time()
51     analysis = tune.run(
52         tune.with_parameters(train_evaluate, X=X_ref, y=y_ref),
53         config=config,
54         metric="mean_accuracy",
55         mode="max",
56         verbose=1,
57         resources_per_trial={"cpu": 1},
```

```

58         max_concurrent_trials=1,
59         raise_on_failed_trial=False,
60     )
61     elapsed_time = time.time() - start
62     print(f"Hyperparameter tuning completed in {elapsed_time:.2f} seconds\n")
63     print("Best hyperparameters found:", analysis.best_config)
64     print("Best mean accuracy:", analysis.best_result["mean_accuracy"])
65
66
67 if __name__ == "__main__":
68     main()

```

A.3 OpenStack Client Configuration

A.3.1 *constants.py*

Configuration constants for OpenStack instance provisioning:

Listing 3: constants.py

```

1 FLAVOR = "ssc.small"
2 PRIVATE_NET = "UPPMAX 2026/1-24 Internal IPv4 Network"
3 IMAGE_NAME = "Ubuntu 22.04 - 2024.01.15"
4 KEY_NAME = "de1-course-snlic-key"

```

A.3.2 *start_instance.py*

OpenStack Nova client script for provisioning Ray cluster instances:

Listing 4: start_instance.py

```

1 # http://docs.openstack.org/developer/python-novaclient/ref/v2/servers.html
2 import time, os, sys, random, re
3 import inspect
4 from os import environ as env
5 from pathlib import Path
6
7 # Add parent directory to sys.path to import constants
8 sys.path.insert(0, str(Path(__file__).parent.parent))
9
10 from novaclient import client
11 import keystoneclient.v3.client as ksclient
12 from keystoneauth1 import loading
13 from keystoneauth1 import session
14 from constants import FLAVOR, PRIVATE_NET, IMAGE_NAME
15
16 flavor = FLAVOR
17 private_net = PRIVATE_NET
18 floating_ip_pool_name = None
19 floating_ip = None
20 image_name = IMAGE_NAME
21
22 identifier = random.randint(1000,9999)
23
24 loader = loading.get_plugin_loader('password')
25
26 auth = loader.load_from_options(auth_url=env['OS_AUTH_URL'],
27                                username=env['OS_USERNAME'],
28                                password=env['OS_PASSWORD'],

```

```
29         project_name=env['OS_PROJECT_NAME'],
30         project_domain_id=env['OS_PROJECT_DOMAIN_ID'],
31         #project_id=env['OS_PROJECT_ID'],
32         user_domain_name=env['OS_USER_DOMAIN_NAME'])
33
34 sess = session.Session(auth=auth)
35 nova = client.Client('2.1', session=sess)
36 print ("user authorization completed.")
37
38 image = nova.glance.find_image(image_name)
39
40 flavor = nova.flavors.find(name=flavor)
41
42 if private_net != None:
43     net = nova.neutron.find_network(private_net)
44     nics = [{'net-id': net.id}]
45 else:
46     sys.exit("private-net not defined.")
47
48 #print("Path at terminal when executing this file")
49 #print(os.getcwd() + "\n")
50 cfg_file_path = os.getcwd()+'/cloud-cfg.txt'
51 if os.path.isfile(cfg_file_path):
52     with open(cfg_file_path) as f:
53         userdata = f.read()
54 else:
55     sys.exit("cloud-cfg.txt is not in current working directory")
56
57 secgroups = ['default']
58
59 print ("Creating instances ... ")
60 # instance = nova.servers.create(name="prod_server_without_docker_"+str(identifier),
61     image=image, key_name='sztoor', flavor=flavor,userdata=userdata,
62     nics=nics,security_groups=secgroups)
63
64 vm_names = ["arnab-a4-ray-head", "arnab-a4-ray-worker-1", "arnab-a4-ray-worker-2"]
65 instances = []
66
67 # craeting 3 instances with the same configuration but different names.
68 for vm_name in vm_names:
69     instance = nova.servers.create(
70         name=vm_name,
71         image=image,
72         flavor=flavor,
73         key_name='de1-course-snic-key',
74         userdata=userdata,
75         nics=nics,
76         security_groups=secgroups
77     )
78     instances.append(instance)
79     print ("waiting for 10 seconds.. ")
80     time.sleep(10)
81
82 for instance in instances:
83     inst_status = instance.status
84     print ("waiting for 10 seconds.. ")
```

```

85     time.sleep(10)
86
87     while inst_status == 'BUILD':
88         print ("Instance: "+instance.name+" is in "+inst_status+" state, sleeping for
89         5 seconds more...")
90         time.sleep(5)
91         instance = nova.servers.get(instance.id)
92         inst_status = instance.status
93
94     ip_address = None
95     for network in instance.networks[private_net]:
96         if re.match('\d+\.\d+\.\d+\.\d+', network):
97             ip_address = network
98             break
99     if ip_address is None:
100         raise RuntimeError('No IP address assigned!')
101
102     print ("Instance: "+ instance.name +" is in " + inst_status + " state" + " ip
103     address: "+ ip_address)

```

A.4 Ansible Playbooks

A.4.1 *install_ray.yml*

Ansible playbook to install Ray and dependencies on cluster nodes:

Listing 5: *install_ray.yml*

```

1 - hosts: raycluster
2   become: true
3   tasks:
4     - name: Update apt
5       apt:
6         update_cache: yes
7         upgrade: dist
8
9     - name: Install pip
10      apt:
11        pkg: python3-pip
12        state: latest
13
14    - name: Install Ray and scikit-learn
15      pip:
16        name:
17          - "ray[tune]"
18          - scikit-learn
19          - numpy
20        executable: pip3

```